

Name: Vũ Đại Dương

ID: 23520359

Class: IT007.P28.2

OPERATING SYSTEM LAB X'S REPORT

SUMMARY

Task		Status	Page
Section 1.5	Exs.1	Done	2 - 5
	Ex.3-1	Done	2
	Ex.3-2	Done	3 - 4
	Ex.3-3	Done	4 - 5
	Ex.3-4	Done	5
	Exs.2	Done	5 - 6
	Exs.3	Done	6
	Exs.4	Done	6 - 8
...	...		
	...		

Self-scores:

Note: Export file to **PDF and name the file by following format:
Student ID_LABx.pdf*

Section 3.5

1. Ex1:

a. VD3-1:

- Giải thích code:

pid = fork(); → tạo tiến trình con

- Tiến trình cha:

```
if (pid > 0)
{
    printf("PARENTS | PID = %ld | PPID = %ld\n",
        (long) getpid(), (long) getppid());
    if (argc > 2)
        printf("PARENTS | There are %d arguments\n", argc - 1);
    wait(NULL);
}
```

- In ra PID và PPID của tiến trình cha.
- Nếu có hơn 1 đối số thì in ra số lượng tham số (argc).
- wait(NULL): đợi tiến trình con kết thúc rồi tiến trình cha mới kết thúc

- Tiến trình con:

```
if (pid == 0)
{
    printf("CHILDREN | PID = %ld | PPID = %ld\n", (long) getpid(), (long) getppid());
    printf("CHILDREN | List of arguments: \n");
    for (int i = 1; i < argc; i++)
    {
        printf("%s\n", argv[i]);
    }
}
```

- + In ra PID và PPID của tiến trình con
- + In ra các tham số truyền vào.

- Giải thích kết quả:

```
● vudaiduong-23520359@VuDuong-32:~$ ./test_fork ThamSo1 ThamSo2 ThamSo3
PARENTS | PID = 1473 | PPID = 768
PARENTS | There are 3 arguments
CHILDREN | PID = 1474 | PPID = 1473
CHILDREN | List of arguments:
ThamSo1
ThamSo2
ThamSo3
```

- Dòng 1: in PID và PPID của tiến trình con
- Dòng 2: in PID và PPID của tiến trình cha
- Dòng 3: in số lượng tham số truyền vào (nếu > 2)
- Các dòng còn lại: in danh sách tham số truyền vào

b. VD3-2:

- Giải thích code:

pid = fork(); → tạo tiến trình con

- Tiến trình cha:

```
if (pid > 0)
{
    printf("PARENTS | PID = %ld | PPID = %ld\n",
(long) getpid(), (long) getppid());
    if (argc > 2)
        printf("PARENTS | There are %d arguments\n",
        argc - 1);
    wait(NULL);
}
```

- o In PID và PPID của tiến trình cha.
- o Nếu có > 2 tham số, in số lượng tham số
- o wait(NULL): đợi tiến trình con kết thúc rồi tiến trình cha mới kết thúc.

- Tiến trình con:

```
if (pid == 0)
{
    execl("./count.sh", "./count.sh", "10", NULL);

    printf("CHILDREN | PID = %ld | PPID = %ld\n",
(long) getpid(), (long) getppid());
    printf("CHILDREN | List of arguments: \n");
    for (int i = 1; i < argc; i++)
    {
        printf("%s\n", argv[i]);
    }
}
```

- o + Gọi script count.sh với đối số "10" (đếm từ 1 đến 10).
- o + Nếu execl() **thành công**, các dòng dưới không chạy.
- o + Nếu execl() **thất bại** (không có file, không có quyền), chương trình in PID và danh sách đối số dòng lệnh.

- Giải thích kết quả:

```
• vudaiduong-23520359@VuDuong-32:~$ ./test_exec1 ThamSo1 ThamSo2 ThamSo3
PARENTS | PID = 5470 | PPID = 5083
PARENTS | There are 3 arguments
CHILDREN | PID = 5471 | PPID = 5470
CHILDREN | List of arguments:
ThamSo1
ThamSo2
ThamSo3
```

- o Dòng 1: in PID và PPID của tiến trình cha
- o Dòng 2: in số lượng tham số
- o *Vì chưa được cấp quyền → execl() thất bại
- o Dòng 3: in PID và PPID của tiến trình con

- Các dòng còn lại: in các tham số truyền vào

- Tạo file count.sh và cấp quyền:

```
vudaaiduong-23520359@VuDuong-32:~$ chmod +x count.sh
```

- Kết quả sau khi cấp quyền:

```
PARENTS | PID = 1154 | PPID = 768
PARENTS | There are 3 arguments
Implementing: ./count.sh
PPID of count.sh:
vudaidu+ 1155 1154 0 13:34 pts/6 00:00:00 /bin/bash ./count.sh 10
vudaidu+ 1157 1155 0 13:34 pts/6 00:00:00 grep count.sh
```

- Dòng 3 trở đi: in kết quả của file count.sh

c. VD3-3:

- Giải thích code:

```
printf("PARENTS | PID = %ld | PPID = %ld\n",
(long)getpid(), (long)getppid());
if (argc > 2)
printf("PARENTS | There are %d arguments\n", argc
- 1);
```

- In PID và PPID của tiến trình cha
- Nếu có > 2 tham số, in số lượng tham số

- system("./count.sh 10"); → thực hiện đoạn Script

```
printf("PARENTS | List of arguments: \n");
for (int i = 1; i < argc; i++)
{
printf("%s\n", argv[i]);
}
```

- In danh sách tham số truyền vào

- Giải thích kết quả:

```
• vudaaiduong-23520359@VuDuong-32:~$ ./test_system ThamSo1 ThamSo2 ThamSo3
PARENTS | PID = 1046 | PPID = 768
PARENTS | There are 3 arguments
Implementing: ./count.sh
PPID of count.sh:
vudaidu+ 1047 1046 0 13:32 pts/6 00:00:00 sh -c -- ./count.sh 10
vudaidu+ 1048 1047 0 13:32 pts/6 00:00:00 /bin/bash ./count.sh 10
vudaidu+ 1050 1048 0 13:32 pts/6 00:00:00 grep count.sh
PARENTS | List of arguments:
ThamSo1
ThamSo2
ThamSo3
```

- Dòng 1: in PID và PPID
- Dòng 2: in số lượng tham số

- Dòng 3 – 7: In đoạn Script
- Các dòng còn lại: in các tham số truyền vào

d. VD3-4:

```

vudaiduong-23520359@VuDuong-32:~$ ./test_shm_A Hello Process B
Waiting Process B update shared memory

vudaiduong-23520359@VuDuong-32:~$ ./test_shm_B Hello Process A
Read shared memory: Hello Process B
Shared memory updated: Hello Process A

```

2. Ex2:

- Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <sys/wait.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <command>\n", argv[0]);
        exit(1);
    }

    struct timeval start, end;
    pid_t pid;

    gettimeofday(&start, NULL);

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        execlp(argv[1], argv[1], NULL);
        // Nếu execlp thất bại
        perror("execlp failed");
        exit(1);
    } else {
        wait(NULL);
        gettimeofday(&end, NULL);

        // Tính thời gian thực thi
        double elapsed = (end.tv_sec - start.tv_sec) +
            (end.tv_usec - start.tv_usec) / 1000000.0;

        printf("Thời gian thực thi: %.5f giây\n", elapsed);
    }

    return 0;
}

```

- Kết quả:

```

vudaiduong-23520359@VuDuong-32:~$ ./time ls
Myweb SJF.c VD.c bai2.sh count.sh fcfs login.time sorted.file test_exec1 test_fork test_shm_A test_shm_B test_system time try_variables.sh
SJF VD Xin bai3.sh count.txt fcfs.c myfolder system.status test_exec1.c test_fork.c test_shm_A.c test_shm_B.c test_system.c time.c ...
Thời gian thực thi: 0.00361 giây

```

3. Ex3:

- Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child_pid = -1;

void handle_sigint(int sig) {
    if (child_pid > 0) {
        kill(child_pid, SIGKILL); // Dừng tiến trình con
        printf("\ncount.sh has stopped\n");
    }
    exit(0);
}

int main() {
    printf("Welcome to IT007, I am 23520359!\n");

    signal(SIGINT, handle_sigint); // Bắt tín hiệu CTRL+C

    child_pid = fork();

    if (child_pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (child_pid == 0) {
        // Tiến trình con thực thi count.sh
        execl("./count.sh", "./count.sh", "120", NULL);
        perror("execl failed");
        exit(1);
    } else {
        wait(NULL);
    }

    return 0;
}
```

- Kết quả:

```
● vudaiduong-23520359@VuDuong-32:~$ ./stop_script
Welcome to IT007, I am 23520359!
Implementing: ./count.sh
PPID of count.sh:
vudaidu+  2211    2210  0 14:51 pts/6    00:00:00 /bin/bash ./count.sh 120
vudaidu+  2213    2211  0 14:51 pts/6    00:00:00 grep count.sh
^C
count.sh has stopped
```

4. Ex4:

- Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/shm.h>
#include <sys/ipc.h>
#include <sys/wait.h>
#include <time.h>

#define BUFFER_SIZE 10

typedef struct {
    int buffer[BUFFER_SIZE];
    int in;
    int out;
    int total;
} SharedBuffer;

int main() {
    srand(time(NULL));

    // Tạo vùng shared memory
    int shm_id = shmget(IPC_PRIVATE, sizeof(SharedBuffer), IPC_CREAT | 0666);
    if (shm_id < 0) {
        perror("shmget failed");
        exit(1);
    }

    SharedBuffer* shared = (SharedBuffer*) shmat(shm_id, NULL, 0);
    shared->in = 0;
    shared->out = 0;
    shared->total = 0;

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
}

```



```

if (pid == 0) {
    // Consumer process
    while (1) {
        if (shared->in != shared->out) {
            int value = shared->buffer[shared->out];
            shared->out = (shared->out + 1) % BUFFER_SIZE;
            shared->total += value;
            printf("Consumer read: %d | Total: %d\n", value, shared->total);

            if (shared->total > 100) {
                break;
            }

            sleep(1);
        }
    }
    exit(0);
} else {
    // Producer process
    while (1) {
        if (shared->total > 100)
            break;
        int value = 10 + rand() % 11; // [10, 20]

        // Kiểm tra nếu buffer chưa đầy
        if (((shared->in + 1) % BUFFER_SIZE) != shared->out) {
            shared->buffer[shared->in] = value;
            shared->in = (shared->in + 1) % BUFFER_SIZE;
            printf("Producer wrote: %d\n", value);

            sleep(1);
        }
    }

    wait(NULL); // Đợi tiến trình con kết thúc

    // Xóa vùng nhớ dùng chung
    shmdt(shared);
    shmctl(shm_id, IPC_RMID, NULL);
}

return 0;
}

```

- Kết quả:

```

vudaiduong-23520359@VuDuong-32:~$ ./pc
Producer wrote: 11
Consumer read: 11 | Total: 11
Producer wrote: 15
Consumer read: 15 | Total: 26
Producer wrote: 12
Consumer read: 12 | Total: 38
Producer wrote: 18
Consumer read: 18 | Total: 56
Producer wrote: 20
Consumer read: 20 | Total: 76
Producer wrote: 18
Consumer read: 18 | Total: 94
Producer wrote: 16
Consumer read: 16 | Total: 110

```